

# Project Delta

Page 1

saathi

Date 24/11/2015.

Project 1. ML

what does this mean?

CBIR  $\rightarrow$  content based image retrieval

(1)

segmentation using clustering.

what does this mean?

(\*) CBIR  $\rightarrow$  locate samples from a database that are akin to a query, relying on content embedded within image.

$\Rightarrow$  Strategy: calculating similarity b/w "compact vectors" by encoding both query & database imgs as global descriptors.

(\*) CLIP, UNICOM, CNNR GEM split  $\rightarrow$  1 semantic representation [512-D, 512-D, 1024-D] per image.

what is "content"?

"sub regions"

Content  $\Rightarrow$  high level semantics extracted from the backbone model.

compressed learned representation

what it includes:-

- 1. which objects are present?
- 2. their visual patterns.
- 3. overall style.
- 4. color distributions.

- 5. feature
- 6. category enc.
- 7. shape & structure
- 8. scene type.
- 9. Relationship b/w subregions.

"ONLY" latent features

(\*) what do these vectors contain?

- 1. object identity
- 2. structural & vis/color attributes
- 3. global composition.
- 4. High level semantics
- 5. implicit local details.

DO NOT contain "bounding boxes" / "per pixel patterns".

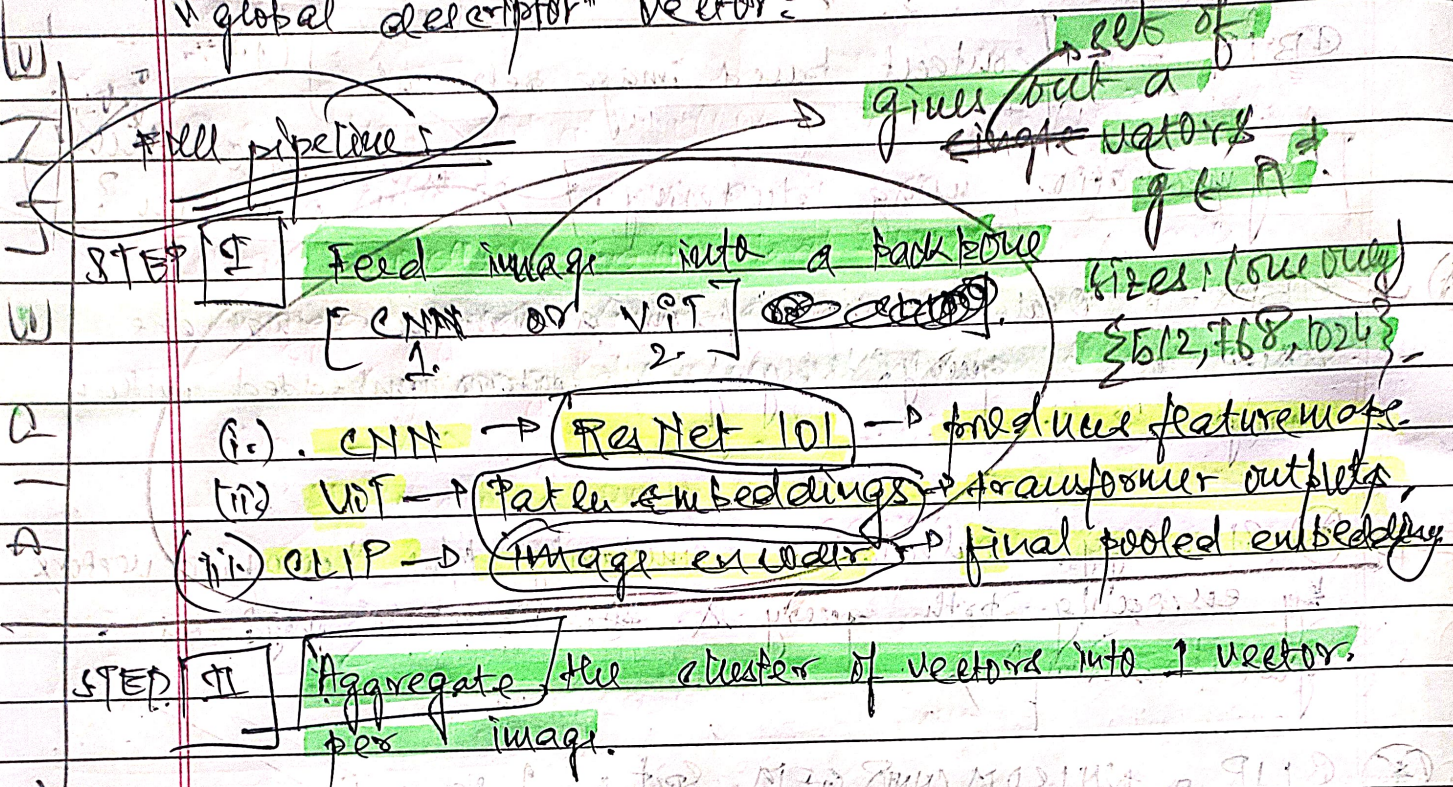


R.P.P

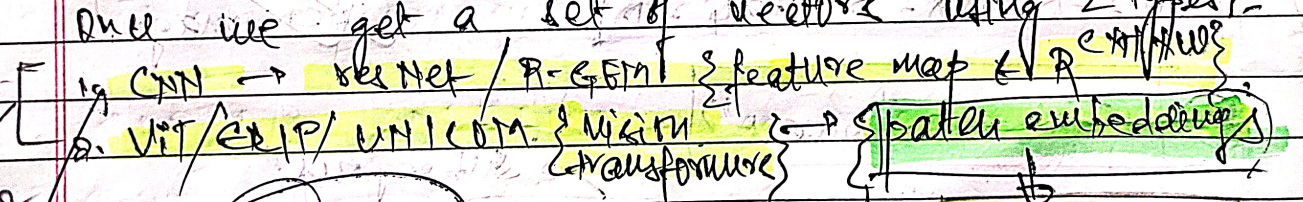
saathi

Date \_\_\_/\_\_\_/\_\_\_

Q) How is a "pxq" image converted into a "global descriptor" vector?



Once we get a set of vectors using 2 types:-



Then aggregation happens:

CASE I: CNN Step → R-GCN

Image:  $(P_1 \times P_2 \times 3)$

C → channel [2048 channels]

H → ?

W → ?

(1) Image separated into  $7 \times 7 = 49$  "regions" (size "regions")

(2) Each portion  $\in \mathbb{R}^c$  dimensional vector representation.

(3) 49 vectors for EACH 2048 size rep vectors.

$F_{i,h,w} \in \mathbb{R}^c \rightarrow \{v_1, v_2, \dots, v_{49}\}$

Aggregation for [1] → Average pool:  $\frac{1}{49} \sum_{h,w} F_{i,h,w}$

Max pool:  $\max_{h,w} \{F_{i,h,w}\}$



These are pre-trained models **Saathi**  
 1) NO learning.  
 2) NO fine-tuning.  
 3) NO weight updates.

~~What is a "QUERY"?~~

## CASE 2: Vision Transformers.

- 1) Image is cut into  $16 \times 16$  /  $14 \times 14$  patches.
- 2)  $16 \times 16 = N \rightarrow$  "sub-images" for EACH img sample.
- 3) Each of these "N" patches are fed into a Vision Transformer:  $\{ \begin{matrix} \text{Patch} \\ \text{Embedding} \end{matrix} \} \rightarrow \text{DNN Transformer} \rightarrow \text{CLS token}$

Image vector representation

~~Each patch embedding  $\{ \begin{matrix} \text{Patch} \\ \text{Embedding} \end{matrix} \} \in \mathbb{R}^D$~~   
 Total number  $\rightarrow N = 16 \times 16$

- (4)  $x_{CLS} \in \mathbb{R}^D$ . prepend it into vector array i.e.  
 $\{ x_{CLS}, x_1^D, x_2^D, \dots, x_N^D \}$

- (5)  $x_{CLS} = f(x_{CLS}, x_1, x_2, \dots, x_N)$   
 $\rightarrow$  Summarizing vector after 'L' layers of transformer

- (6) CLIP, UNICOM  $\rightarrow x_{CLS}^D$  of size D for 1 image.
- (7) Aggregation NOT NECESSARY.  $x_{CLS}$  is SMART.

Now for a dataset of size  $\Delta$  we have  $\Delta$  representation vectors for each image

STEP III. we have descriptor set for each constituent image within leaf nodes.

vectors for each image

~~NOTE: what the user is a QUERY?~~  
 It is an image that asks:  
 "Is this tree/car/animal in an image of your dataset", "can you tell which one?", "which one do you think matches the closest description?"

Next step: Generate query descriptor regularly using STEP I & STEP II.

APPENDIX



~~Now we~~  
~~STEP IV:~~

~~Use the global descriptor of QUERY ( $q$ ) & calculate similarity ( $sq$ ) with leaf nodes. Each leaf node of each image gives some similarity.~~

~~Similarity distance matrix /  $\{sq_1, sq_2, \dots, sq_N\}$  Dataset size.~~

~~$\{v_1, v_2, v_3, \dots, v_N\}$~~   
 ~~$N$  leaves~~

~~$\{s_1, s_2, s_3, \dots, s_N\}$~~   
~~scalars~~

~~Speed Accuracy Tradeoff~~

STEP IV:

OFFLINE stage.

$N$  - stored dataset

→ Now we have to understand that we start with OFFLINE stage.

→ FOR CNN path → aggregated info rich vector  
 FOR ViT path → all vector.

→ Now we have  $N$  such vectors.

→ Implement HKM → Hierarchical K-means.  
 AND build search TREE.

→ Root node →  $K$  children.  $\{K$  centroids.

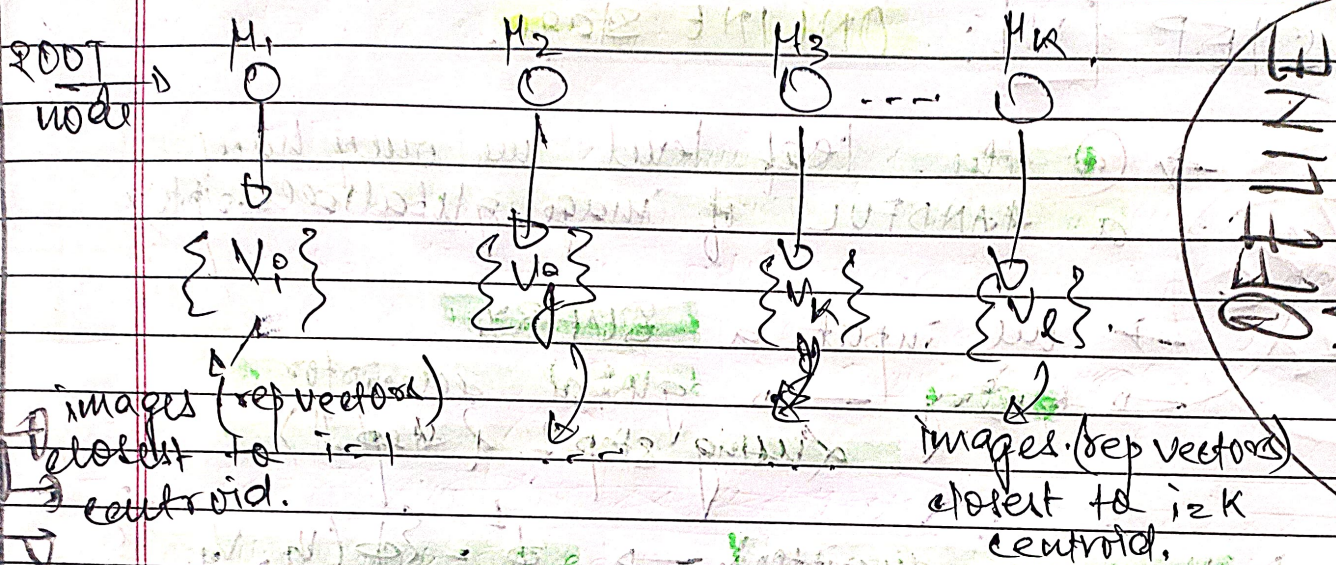
→ Each  $i^{th}$  node has images which are closest to the  $i^{th}$  centroid.  $\{K = N\}$ .



Hyperparameter: 1.  $K \Rightarrow K$ -means  $\rightarrow$  branching factor (AKA).  
 2.  $l \Rightarrow$  tree depth.

Page 5  
 Saathi

Date \_\_\_ / \_\_\_ / \_\_\_



INITIAL  
 FINAL  
 process

Run Iterative  $K$ -means and correctly bucket images. After several  $K$ -means, we finally get the correct buckets (final  $K$ -buckets). Each bucket has  $\sum b_i$  number of images.

$$\sum_{i=1}^m \delta_i^{(1)} = N$$

For  $l > 2$ , we do  $K$ -means again, again get  $K$  buckets for each child.

$$\sum_{j=1}^m \sum_{k=1}^K b_{ij,k}^{(2)} = \delta_i^{(1)} \Rightarrow \sum_{k=1}^K \sum_{j=1}^m b_{ij,k}^{(2)} = \delta_i^{(1)}$$

$$\sum_{\alpha=1}^m \sum_{k=1}^K b_{ijk,\alpha}^{(3)} = b_{ijk}^{(2)} \Rightarrow \sum_{\alpha=1}^m b_{ijk,\alpha}^{(3)} = b_{ijk}^{(2)}$$

The recursion is dependent upon "l" leaf/tree depth.

Each of these buckets at any depth will only contain A VERY SMALL subset of images.

Leaf nodes  $\rightarrow$  store ACTUAL image vectors  
 INTERNAL nodes  $\rightarrow$  "label"  $\rightarrow$  centroid vectors @ each node.



PG

saathi

Date \_\_\_ / \_\_\_ / \_\_\_

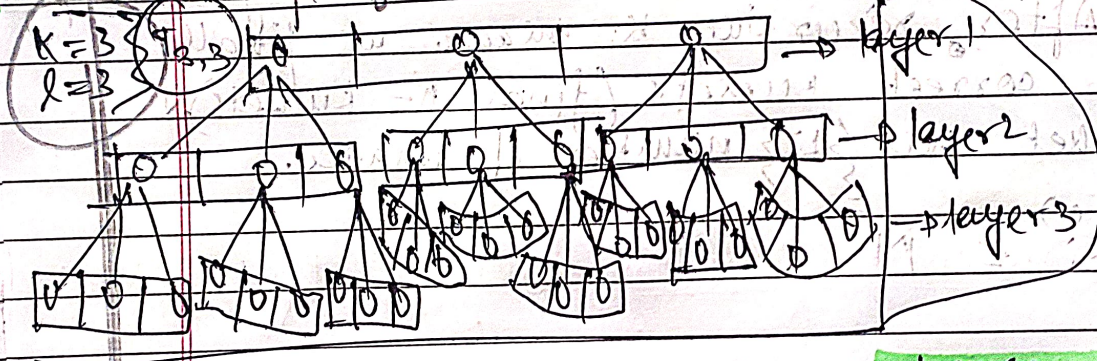
# STEP [V]: ONLINE stage.

ONLY ONE cell

→ @ the leaf level we only have a HANDFUL of image global descriptors

→ we input a QUERY.  
→ Query → global descriptor (using step I & step II),

→  $V_q$  (query descriptor) →  $\phi = \{\phi(V_q, V_{p_k})\}$  layer 1



choose argmin( $\phi^1$ )

choose argmin( $\phi^1$ ) → move on to layer 2

→ move over to layer 3 → choose argmin( $\phi^2$ )

→ ~~Now~~ Now the similarity metrics were b/w  $V_q$  &  $V_{p_k}$ . Now that we have THE best Node.  $p_k$  we choose THE candidate images in THAT node & calculate similarity vector.

→ Suppose BEST node has  $\beta$  images, we calculate pair wise similarity →  $\{s_1, s_2, s_3, \dots, s_{\beta}\}$ .  
AND then sort these images.



Saathi

Date \_\_\_/\_\_\_/\_\_\_

→ sorted  $\{s_0, s_1, s_2, s_3, \dots, s_{q-1}\} \Rightarrow$  RANKING

[THAT'S IT.] Now,  $\text{argmax}[\text{sorted } \{s_i\}] = \text{ANS}$

answer